

System Monitorowania Modelu ML

Autor: Ewelina Tołwińska

1. Wstęp

Wdrożenie modelu uczenia maszynowego do środowiska produkcyjnego nie stanowi końca procesu wytwórczego, lecz inicjuje fazę operacyjną, w której model poddawany jest weryfikacji przez rzeczywiste dane.

W przypadku aplikacji działających w oparciu o uczenie maszynowe, tradycyjne systemy monitorowania wydajności aplikacji okazują się niewystarczające. Współczesne metodologie monitoringu systemów, począwszy od *Four Golden Signals* opracowanych w Google w ramach metodologii SRE (*Site Reliability Engineering*), przez metodę RED (*Rate, Errors, Duration*) dedykowaną architekturze mikrousługowej, aż po metodę USE (*Utilization, Saturation, Errors*) skupioną na infrastrukturze, koncentrują się na stabilności i technicznej poprawności serwisu. Narzędzia operacjonalizujące te metodologie (rozwiązania klasy *Application Performance Monitoring*) skutecznie odpowiadają na pytanie „czy usługa działa?”, jednak pozostają skupione wyłącznie na awariach infrastrukturalno-aplikacyjnych.

W swojej podstawowej formie nie dostarczają one metryk pozwalających na wykrywanie awarii specyficznych dla systemów uczenia maszynowego (*ML-specific failures*, Huyen, 2022). Mowa tu m.in. o zjawisku cichej degradacji modelu (*silent failure*), w której system zwraca odpowiedzi poprawne pod kątem technicznym, ale przestaje odzwierciedlać rzeczywistość, ponieważ dane wejściowe uległy zmianie względem danych treningowych.

W przypadku usług z wdrożonym modelem ML odpowiedź na pytanie „czy usługa działa?” wymaga zatem weryfikacji szeregu dodatkowych elementów, wykraczających poza klasyczne metryki. Sama możliwość przeprowadzenia takiej weryfikacji nierzadko wymusza zmiany w architekturze systemu - przykładowo wprowadzenie dedykowanych mechanizmów zbierania rzeczywistych wartości docelowych (*ground truth*) wraz z odpowiadającą im pętlą zwrotną (*feedback loop*), bez której ocena trafności predykcji modelu w czasie pozostaje niemożliwa.

Obserwowalność systemu ML staje się tym samym nie tylko zagadnieniem monitoringu, lecz również świadomego projektowania architektury, uwzględniającego od początku potrzebę gromadzenia, korelacji i analizy danych wykraczających poza klasyczną telemetrię infrastrukturalną.

2. Cel i zakres projektu

Niniejszy projekt podejmuje temat monitorowania modeli uczenia maszynowego w środowisku produkcyjnym, ze szczególnym uwzględnieniem zjawisk degradacji modelu w czasie. Zakres prac obejmuje dwa etapy.

- **Etap teoretyczny** obejmuje identyfikację i klasyfikację problemów produkcyjnych prowadzących do degradacji modeli uczenia maszynowego - ze szczególnym uwzględnieniem zjawisk dryfu danych (*covariate drift*), dryfu etykiet (*label shift*) oraz dryfu koncepcji (*concept drift*) - wraz z analizą metryk pozwalających na ich detekcję.
- **Etap praktyczny** obejmuje zaprojektowanie i implementację systemu monitoringu operacjonalizującego wnioski z etapu teoretycznego. W ramach tego etapu zaimplementowano model regresji aproksymujący funkcję sinusoidalną postaci $A \cdot \sin(B \cdot x) + \epsilon$, gdzie ϵ oznacza składnik losowy, udostępniony jako usługa REST API, zintegrowany z systemem monitoringu łączącym klasyczne narzędzia monitorowania infrastruktury - Prometheus, Grafana - z biblioteką dedykowaną analizie dryfu danych Evidently AI. Wybór funkcji sinusoidalnej jako modelowanego zjawiska jest celowy. Jej kontrolowalność i przewidywalność umożliwiają powtarzalne indukowanie poszczególnych typów problemów produkcyjnych związanych z dryfem danych w środowisku eksperymentalnym.

Projekt ma charakter *proof of concept* (PoC) i kładzie nacisk na aspekt edukacyjno-demonstracyjny - jego celem jest zaprezentowanie mechanizmów monitoringu modeli uczenia maszynowego, nie zaś rozwiązanie konkretnego problemu biznesowego.

3. Problemy produkcyjne modeli uczenia maszynowego

3.1. Problemy infrastrukturalno-aplikacyjne (Software System Failures)

Klasa problemów, które występują również w klasycznych systemach informatycznych, niezwiązanych bezpośrednio z modelami uczenia maszynowego, lecz z warstwą techniczną, w której działa.

Należą do nich zjawiska znane z eksploatacji systemów: zwiększona latencja odpowiedzi, błędy serwera (np. odpowiedzi HTTP 5xx), niedostępność usługi, wyczerpanie zasobów sprzętowych (CPU, RAM, GPU), problemy z pamięcią masową, błędy w komunikacji sieciowej, niepoprawne wdrożenie nowej wersji aplikacji czy awarie usług zewnętrznych, od których system jest zależny. Do tej samej kategorii zaliczyć można również błędy w potokach danych - nieprawidłowe łączenie danych z wielu źródeł, błędne typy danych, brak części danych, czy niespójności schematów.

Problemy te są stosunkowo dobrze rozpoznane w inżynierii oprogramowania, a ich detekcja stanowi domenę klasycznych metodologii monitoringu. Mimo, że nie są one specyficzne dla systemów uczenia maszynowego, stanowią dużą część rzeczywistych awarii produkcyjnych systemów ML.

3.2. Problemy specyficzne dla uczenia maszynowego (ML-Specific Failures)

Ta klasa obejmuje problemy występujące wyłącznie w systemach opartych na uczeniu maszynowym, wynikające z natury samego modelu. W odróżnieniu od problemów infrastrukturalno-aplikacyjnych, ich symptomy zazwyczaj nie objawiają się w klasycznych metrykach technicznych. Model nadal odpowiada w wymaganym czasie, zwracając wyniki poprawne syntaktycznie, lecz jego skuteczność predykcyjna ulega degradacji.

Spektrum tego typu problemów jest szerokie i obejmuje m.in. zdegenerowane pętle zwrotne (ang. *degenerate feedback loops*), przypadki brzegowe (ang. *edge cases*), degradację kalibracji predykcji czy podatność na dane adversarialne (Huyen, 2022). W niniejszej pracy skupiono się na zjawisku **dryfu rozkładu danych** (ang. *data distribution shift*) obejmującym zmiany w rozkładzie danych wejściowych, etykiet

oraz relacji między nimi jako jednego ze źródeł degradacji modeli w środowisku produkcyjnym, możliwego do zautomatyzowanego wykrycia przy użyciu metod statystycznych.

Dryf rozkładu danych (ang. *data distribution shift*) stanowi zjawisko polegające na tym, że rozkład danych, na których model wykonuje predykcje w środowisku produkcyjnym, zmienia się w czasie względem rozkładu danych, na których model został wytrenowany. Następstwem tego zjawiska możliwa jest stopniowa degradacja skuteczności predykcyjnej modelu. Poniżej omówione zostały podstawowe typy dryfu danych wyróżniane w literaturze.

3.2.1. Dryf danych wejściowych (covariate shift)

Dryf danych wejściowych zachodzi, gdy rozkład zmiennych wejściowych $P(X)$ ulega zmianie, podczas gdy rozkład warunkowy $P(Y|X)$ pozostaje stały. Oznacza to, że zmienia się charakterystyka danych obserwowanych przez model, lecz relacja pomiędzy wejściem a wyjściem pozostaje niezmienną.

Przykładem może być model predykcyjny estymujący czas dostawy zamówienia, wykorzystujący odległość pomiędzy źródłem a miejscem docelowym dostawy jako jedną z głównych cech wejściowych. Model został wytrenowany w okresie, w którym serwis operował głównie w aglomeracji miejskiej. Rozkład cechy w danych treningowych koncentrował się na bliskich odległościach - w okolicy 5 km. Po wprowadzeniu usługi poza granice aglomeracji, rozkład odległości w danych produkcyjnych uległ zmianie, obejmując także wyższe wartości. Charakterystyka rozkładu wejściowego cechy odległość uległa przesunięciu, lecz relacja pomiędzy odległością a czasem dostawy pozostała niezmienną. Dostawa na 25 km nadal trwa średnio tyle samo czasu, niezależnie od tego, czy adres znajduje się w aglomeracji, czy poza nią. Należy podkreślić istotną właściwość dryfu danych wejściowych, sam fakt jego wystąpienia nie oznacza automatycznie degradacji jakości predykcji modelu. Jeśli zmiana rozkładu zachodzi w obszarze dobrze reprezentowanym przez zbiór treningowy, model nadal stosuje wyuczoną aproksymację w punktach, dla których jest ona prawdziwa, zachowując swoją skuteczność. Dryf danych wejściowych pełni zatem rolę sygnału ostrzegawczego, wskazującego na zmianę w środowisku operacyjnym, lecz nie stanowi bezpośredniego dowodu na spadek jakości modelu.

3.2.2. Dryf etykiet (label shift)

Dryf etykiet zachodzi, gdy rozkład etykiet $P(Y)$ ulega zmianie, podczas gdy rozkład warunkowy $P(X|Y)$ pozostaje stały. Innymi słowy: zmienia się proporcja klas lub rozkład wartości docelowej, lecz dla danej klasy charakterystyka cech wejściowych pozostaje niezmienniona.

Przykładem może być klasyfikator wiadomości elektronicznych rozpoznający spam. W okresie wzmożonej aktywności kampanii phishingowych udział wiadomości będących spamem w ogólnym strumieniu poczty istotnie wzrasta, co stanowi zmianę rozkładu etykiet $P(Y)$. Jednocześnie charakterystyka samych wiadomości spamowych, tzn. typowe słowa kluczowe, struktura wiadomości, cechy nadawcy pozostaje statystycznie taka sama i wiadomość będąca spamem nadal wygląda jak spam.

3.2.3. Dryf koncepcji (concept drift)

Dryf koncepcji zachodzi, gdy rozkład warunkowy $P(Y|X)$ ulega zmianie, podczas gdy rozkład wejściowy $P(X)$ pozostaje stały. Dryf koncepcji stanowi szczególnie groźną klasę problemów produkcyjnych z dwóch zasadniczych powodów. Po pierwsze, jest niewykrywalny przez monitoring rozkładu danych wejściowych. Z definicji $P(X)$ pozostaje niezmiennione, w związku z czym żadna metryka statystyczna porównująca rozkłady wejściowe nie zasygnalizuje problemu. Po drugie, jego detekcja wymaga porównania predykcji modelu z rzeczywistymi wartościami docelowymi, a tym samym dostępu do ground truth, który w warunkach produkcyjnych bywa opóźniony lub w ogóle niedostępny. W konsekwencji dryf koncepcji może działać niezauważony przez długi czas, prowadząc do podejmowania błędnych decyzji biznesowych na podstawie nieaktualnej już aproksymacji rzeczywistości.

Przykładem może być model prognozujący ceny mieszkań w Polsce, oparty na cechach takich jak metraż, lokalizacja, liczba pokoi, piętro czy rok budowy. Model wytrenowany został na danych z lat 2019–2021. W okresie 2022–2023 na skutek wydarzeń na Ukrainie relacja pomiędzy cechami mieszkania a jego ceną uległa istotnej zmianie. Mieszkanie o tej samej charakterystyce, które w 2021 roku kosztowało 450 000 PLN, w 2023 roku osiągało cenę 600 000 PLN, mimo że rozkład

samych cech mieszkań na rynku pozostał statystycznie zbliżony. Model nadal otrzymuje na wejściu podobne dane, lecz prawidłowa odpowiedź zmieniła się. Warunkowy rozkład ceny przy danych cechach mieszkania uległ przesunięciu.

3.2.4. Współwystępowanie typów dryfu

Należy zaznaczyć, że w warunkach produkcyjnych poszczególne typy dryfu rzadko występują w izolacji. Pojedyncza zmiana w środowisku operacyjnym może jednocześnie skutkować dryfem danych wejściowych, dryfem etykiet oraz dryfem koncepcji.

3.2.5 Charakter czasowy dryfu

Niezależnie od typu mechanizmu (dryf danych wejściowych, etykiet lub koncepcji), dryf rozkładu danych może charakteryzować się różną dynamiką czasową, występując w sposób nagły, stopniowy lub cykliczny. Wymiar ten jest ortogonalny do klasyfikacji mechanizmu i każdy z trzech wcześniej omówionych typów dryfu może wystąpić w dowolnej z wymienionych dynamik.

Charakter czasowy dryfu stanowi dodatkowe wyzwanie w projektowaniu systemu monitoringu, w szczególności w zakresie odpowiedniego doboru okna analizy oraz definicji rozkładu referencyjnego, względem którego prowadzone są porównania. Szczegółowa analiza poszczególnych strategii detekcji dla różnych dynamik dryfu pozostaje poza zakresem niniejszej pracy.

4. Architektura systemu monitoringu

Po zdefiniowaniu typów dryfu rozkładu danych przedmiotem dalszych rozważań jest praktyczna realizacja systemu umożliwiającego ich detekcję i wizualizację w czasie rzeczywistym. Niniejszy rozdział omawia założenia projektowe, dobór stosu technologicznego oraz architekturę zaproponowanego rozwiązania.

4.1. Założenia projektowe

System został zaprojektowany jako prototyp typu *proof of concept*, kładący nacisk na aspekt edukacyjno-demonstracyjny. Celem projektu jest zaprezentowanie mechanizmów monitoringu modeli uczenia maszynowego, nie zaś rozwiązanie konkretnego problemu biznesowego. Z założeniami tymi związany jest dobór modelowanego zjawiska, którym jest zaszumiona funkcja sinusoidalna postaci $A \cdot \sin(B \cdot x) + \varepsilon$, gdzie ε oznacza składnik losowy. Wybór ten stanowi świadomą decyzję metodologiczną. Kontrolowalność i przewidywalność tej funkcji pozwala na powtarzalne i kontrolowane wprowadzanie poszczególnych typów dryfu w środowisku eksperymentalnym, co w przypadku zbiorów danych pochodzących z rzeczywistych systemów produkcyjnych byłoby trudne do osiągnięcia.

Z charakteru projektu wynika również szereg świadomych ograniczeń zakresu. Z systemu wyłączono obsługę opóźnionego ground truth (ang. *delayed labels*). Wartości docelowe obliczane są analitycznie z wykorzystaniem znajomości funkcji generującej dane, co eliminuje potrzebę implementacji pełnej pętli zwrotnej (ang. *feedback loop*). Środowisko eksperymentalne zostało przygotowane przy użyciu narzędzia Docker.

4.2. Stos technologiczny

Stos technologiczny systemu zaprojektowano w oparciu o rozwiązania open source, stanowiące standard branżowy w obszarze monitoringu systemów produkcyjnych oraz monitoringu modeli uczenia maszynowego.

Model uczenia maszynowego zaimplementowano w bibliotece *scikit-learn* z wykorzystaniem klasy *MLPRegressor* - wielowarstwowego perceptronu (ang. *multilayer perceptron*) realizującego zadanie regresji. Wybór klasycznej, prostej architektury sieciowej podyktowany jest charakterem zadania. Aproksymacja funkcji sinusoidalnej nie wymaga zaawansowanych struktur.

Warstwę serwowania modelu stanowi aplikacja zbudowana w oparciu o framework *FastAPI*, udostępniająca model jako usługę REST API. FastAPI wybrano ze względu na wbudowane wsparcie dla asynchronicznego przetwarzania żądań, natywną

obsługę specyfikacji OpenAPI oraz dobrą integrację z ekosystemem narzędzi obserwowalności.

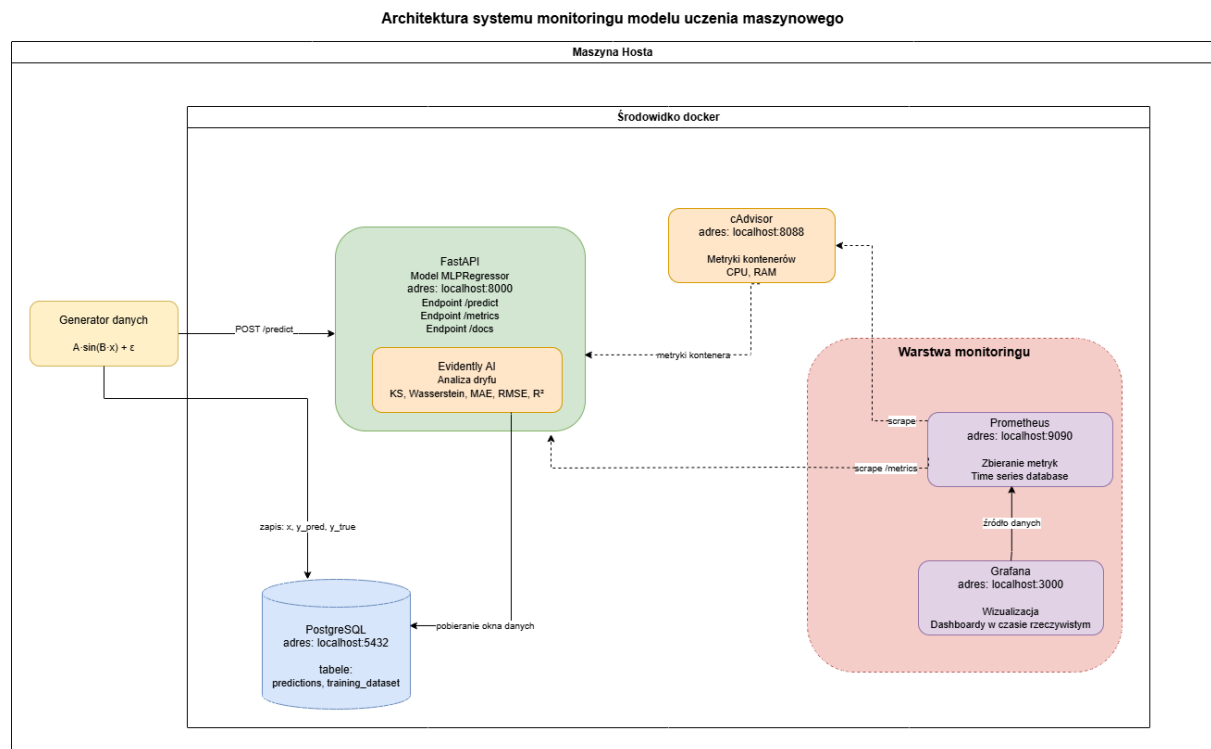
Warstwę przechowywania danych realizuje system bazodanowy *PostgreSQL*, służący do zapisu predykcji modelu wraz z odpowiadającymi im wartościami wejściowymi i wartościami docelowymi. Persystencja danych umożliwia obliczanie metryk dryfu na oknach czasowych oraz historyczną analizę zachowania modelu.

Monitoring infrastrukturalny zrealizowano przy użyciu pary narzędzi *Prometheus* i *Grafana*. Prometheus pełni rolę systemu zbierania i przechowywania metryk w postaci szeregów czasowych, pobierając dane z dedykowanych endpointów aplikacji w określonych interwałach. Grafana stanowi warstwę wizualizacji, prezentującą zgromadzone metryki w postaci interaktywnych dashboardów.

Monitoring dryfu danych i modelu realizowany jest przy użyciu biblioteki *Evidently AI*. Jest to rozwiązanie open source dedykowanego analizie statystycznej rozkładów danych oraz jakości predykcji w systemach uczenia maszynowego. Evidently dostarcza implementacji szeregu testów statystycznych służących do porównywania rozkładów (test Kołmogorowa-Smirnowa, dystans Wassersteina) oraz klasycznych metryk skuteczności modelu w zadaniach regresji (średni błąd bezwzględny, średni błąd kwadratowy, współczynnik determinacji). Pozwala to na monitorowanie zarówno dryfu rozkładu danych wejściowych i predykcji, jak i bezpośredniej degradacji jakości modelu względem wartości docelowych.

Środowisko uruchomieniowe stanowi zestaw kontenerów *Docker*, orkiestrowanych przez *Docker Compose*. Konteneryzacja zapewnia powtarzalność uruchomienia całego systemu na dowolnej maszynie wspierającej Docker oraz upraszcza zarządzanie zależnościami pomiędzy komponentami.

4.3. Diagram architektury systemu monitoringu



Rysunek 1. Architektura systemu monitoringu wdrożonego modelu uczenia maszynowego

5. Monitorowane metryki

W zaprojektowanym systemie monitoruje się trzy uzupełniające się grupy metryk, odpowiadające trzem warstwom obserwowalności systemu uczenia maszynowego w produkcji: warstwie infrastruktury, warstwie danych wejściowych przetwarzanych przez model oraz warstwie samego modelu. Każda z grup odpowiada na inne pytanie, wykorzystuje inne narzędzia i pozwala wykryć inną klasę problemów produkcyjnych. Wspólnie tworzą one warstwę obserwowalności systemu, integrującą klasyczne podejście do monitoringu infrastruktury z dedykowanymi metodami monitoringu modeli uczenia maszynowego.

5.1. Metryki warstwy infrastruktury

Pierwsza grupa obejmuje klasyczne metryki monitoringu infrastruktury aplikacyjnej, odpowiadające na pytanie „czy usługa działa technicznie poprawnie?”. Metryki tej warstwy zbierane są przez Prometheus z dedykowanego endpointu `/metrics`

udostępnianego przez usługę serwującą model (FastAPI) i prezentowane na dashboardach Grafany.

W zakresie tej warstwy monitoruje się czas odpowiedzi usługi (ang. latency) w postaci kwantyla p95, liczbę obsługiwanych żądań w jednostce czasu (ang. throughput, mierzona w zapytaniach na sekundę) oraz wykorzystanie zasobów obliczeniowych (CPU i pamięci RAM). Wartości te odpowiadają klasycznym sygnałom monitoringu opisanym w metodologii Four Golden Signals oraz RED, omówionym na początku niniejszej pracy.

Warstwa infrastruktury wykrywa problemy z klasy Software System Failures. Nie dostarcza natomiast informacji o jakości predykcji modelu ani o charakterystyce statystycznej przetwarzanych danych. Pełni rolę fundamentu obserwowalności, ponad którym budowane są kolejne warstwy.

5.2. Metryki warstwy danych wejściowych

Druga grupa obejmuje metryki opisujące charakterystykę statystyczną danych trafiających do modelu w fazie wnioskowania. Odpowiadają one na pytanie „czy dane, które model obecnie przetwarza, są zbliżone do danych, na których model był trenowany?”. Punktem odniesienia są dane treningowe z którymi porównuje się rozkład danych obserwowanych w czasie rzeczywistym.

Istotną własnością metryk tej warstwy jest fakt, iż ich obliczenie nie wymaga znajomości wartości docelowej (ang. ground truth). Opierają się one wyłącznie na danych wejściowych modelu, dostępnych natychmiast w momencie wykonywania predykcji. Czyni to warstwę danych wejściowych pierwszą linią obrony przed cichą degradacją modelu, dostarczającą sygnałów ostrzegawczych w czasie rzeczywistym.

Monitorowanie rozkładu cechy wejściowej zachodzi przy użyciu następujących testów:

- **Test Kołmogorowa-Smirnowa** - nieparametryczny test statystyczny porównujący dystrybuanty empiryczne dwóch próbek. W kontekście monitoringu dryfu test ten odpowiada na pytanie, czy obserwowana różnica

między rozkładem bieżącym a referencyjnym jest istotna statystycznie. Wynik testu interpretuje się przez wartość statystyki D oraz odpowiadającą jej wartość p (próg istotności w systemie ustalono na $p < 0.05$).

- **Dystans Wassersteina** - miara odległości pomiędzy dwoma rozkładami prawdopodobieństwa, intuicyjnie odpowiadająca minimalnemu kosztowi „przetransportowania” masy probabilistycznej z jednego rozkładu do drugiego. W odróżnieniu od testu KS, dystans Wassersteina dostarcza liczbową miarę skali rozbieżności, wyrażonej w jednostkach monitorowanej zmiennej. Czyni go to użytecznym w definiowaniu progów alertów oraz interpretacji rozmiaru zmiany.

Zastosowanie obu testów dostarcza pełniejszego obrazu zmian rozkładu wejściowego niż każdy z nich osobno. Test KS sygnalizuje, czy zmiana jest istotna statystycznie, natomiast dystans Wassersteina mierzy o ile rozkłady się od siebie różnią. Metryki warstwy danych wejściowych pozwalają na detekcję dryfu danych wejściowych.

5.3. Metryki warstwy modelu

Trzecia grupa obejmuje metryki opisujące zachowanie samego modelu oraz zgodność jego predykcji z rzeczywistymi wartościami docelowymi. Odpowiadają one na pytanie „czy model nadal przewiduje poprawnie?”.

Monitoring jakości predykcji obejmuje bezpośrednie porównanie wyników modelu z rzeczywistymi wartościami docelowymi za pomocą klasycznych metryk skuteczności w zadaniach regresji:

- **średni błąd bezwzględny** (ang. Mean Absolute Error, MAE),
- **pierwiastek średniego błędu kwadratowego** (ang. Root Mean Squared Error, RMSE),
- **współczynnik determinacji** (R^2).

Metryki te pozwalają bezpośrednio wykryć dryf koncepcji (concept drift), czyli sytuację, w której model przestaje poprawnie odwzorowywać relację pomiędzy cechami a zmienną docelową. Spadek skuteczności predykcji modelu, a zatem

wzrost MAE i RMSE i spadek R^2 sygnalizuje, że relacja w środowisku operacyjnym uległa zmianie względem warunków treningowych.

Istotnym ograniczeniem warstwy jakości predykcji jest zależność od dostępności wartości docelowej. W warunkach produkcyjnych ground truth bywa dostępne ze znacznym opóźnieniem (ang. delayed labels) lub niedostępne w ogóle, co wymaga implementacji dedykowanych mechanizmów zbierania wartości docelowych, tzw. pętli zwrotnej (ang. feedback loop). W ramach prezentowanego prototypu wartości docelowe obliczane są analitycznie z wykorzystaniem znajomości funkcji generującej dane, co stanowi świadome uproszczenie podyktowane charakterem proof of concept.

5.4. Mapowanie metryk na typy problemów produkcyjnych

Powiązanie poszczególnych metryk z typami problemów produkcyjnych modeli uczenia maszynowego, zdefiniowanymi w rozdziale 3, podsumowuje poniższa tabela.

Warstwa	Metryka	Typ problemu	Wymaga ground truth
Infrastruktura	Wykorzystanie CPU	Przeciążenie zasobów obliczeniowych	Nie
Infrastruktura	Wykorzystanie pamięci RAM	Wycieki pamięci, niedoszacowanie zasobów	Nie
Infrastruktura	Czas odpowiedzi [s] (p95)	Wzrost opóźnień, degradacja wydajności	Nie
Infrastruktura	Liczba zapytań na sekundę [req/s]	Anomalie ruchu, niedostępność usługi	Nie
Dane wejściowe	Test Kołmogorowa-Smirnowa	Dryf danych wejściowych (istotność statystyczna)	Nie
Dane wejściowe	Dystans Wassersteina	Dryf danych wejściowych (skala zmiany)	Nie
Model	MAE	Dryf koncepcji / degradacja jakości	Tak

Model	RMSE	Dryf koncepcji / degradacja jakości	Tak
Model	R ²	Dryf koncepcji / degradacja jakości	Tak

Tabela 1. Mapowanie monitorowanych metryk na klasy problemów produkcyjnych modeli uczenia maszynowego.

6. Podsumowanie

Wdrożenie modelu uczenia maszynowego do środowiska produkcyjnego nie stanowi końca procesu wytwórczego, lecz inicjuje fazę operacyjną, w której model poddawany jest weryfikacji przez rzeczywiste dane.

W warstwie teoretycznej usystematyzowano problemy produkcyjne modeli ML, wskazując dryf rozkładu danych jako jedno z głównych źródeł cichej degradacji. Przedstawiono klasyfikację trzech podstawowych typów dryfu wraz z analizą ich wykrywalności.

W warstwie praktycznej zaimplementowano system monitoringu integrujący klasyczne narzędzia obserwowalności infrastrukturalnej (Prometheus, Grafana, cAdvisor) z biblioteką dedykowaną analizie dryfu (Evidently AI). System obejmuje trzy uzupełniające się warstwy obserwowalności: infrastruktury, danych wejściowych oraz samego modelu.

Praca pozwoliła zweryfikować dwie istotne tezy. Po pierwsze, dryf danych wejściowych nie zawsze prowadzi do degradacji jakości predykcji - pełni rolę sygnału ostrzegawczego, a nie dowodu spadku skuteczności modelu. Po drugie, dryf koncepcji pozostaje niewidoczny dla monitoringu rozkładu wejść i wymaga osobnej warstwy monitorowania jakości predykcji, opartej o dostęp do wartości docelowych. Wnioski te potwierdzają konieczność stosowania wielowarstwowego podejścia do monitoringu modeli ML.

Praca pokazuje również, że nawet dla tak prostego zadania jak regresja jednowymiarowej funkcji sinusoidalnej zaprojektowanie efektywnego systemu monitoringu wymagało znacznej infrastruktury, m.in. usługi serwującej model, bazy

danych, dwóch niezależnych warstw zbierania metryk, komponentu analizy statystycznej oraz warstwy wizualizacji. W realnych systemach produkcyjnych, operujących na danych wielowymiarowych i zmiennych w czasie, złożoność tej infrastruktury rośnie nieproporcjonalnie. Z tego względu zaprezentowane rozwiązanie należy traktować jako załączek systemu monitoringu ML, nie zaś gotową architekturę produkcyjną.

Monitoring modeli uczenia maszynowego stanowi szerokie i intensywnie rozwijające się zagadnienie. Niniejsza praca zarysowała jego fundamentalne komponenty na przykładzie kontrolowanego prototypu, jednakże mając na uwadze, że pełne zaadresowanie problemu wymaga znacznie szerszego zakresu prac.

7. Bibliografia

1. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (red.). (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. <https://sre.google/sre-book/monitoring-distributed-systems/>
2. Huyen, C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media.
3. dos Reis, D. M., Flach, P., Matwin, S., & Batista, G. (2016). Fast unsupervised online drift detection using incremental Kolmogorov-Smirnov test. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1545–1554. <https://doi.org/10.1145/2939672.2939836>
4. Evidently AI. (2024). *Data drift detection methods*. Dokumentacja Evidently AI. https://docs.evidentlyai.com/metrics/explainer_drift#data-drift
5. Klaise, J., Van Looveren, A., Cox, C., Vacanti, G., & Coca, A. (2020). Monitoring and explainability of models in production. *ICML 2020 Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*. <https://arxiv.org/abs/2007.06299>